

SOC/L1 -13.1. Log Analysis with SIEM

Task 1 - Introduction

SIEM solutions play a vital role in every Security Operations Centre, and any SOC analyst's day-to-day life.

Task 2 - Benefits of SIEM for Analysts

Centralisation: One of the first things that makes SIEM so helpful for a SOC is centralisation. Instead of checking logs in different places, like network devices, cloud services, identity providers, and more, a SIEM allows you to gather all that data in one place. This means an analyst doesn't have to switch between systems during an investigation. Everything is available in a single solution, making their work much smoother and more efficient.

Correlation: Another core strength of SIEM is correlation, the ability to link separate events and piece them together like pieces of a puzzle to form a complete picture. Let's walk through a scenario. You receive an alert in your SIEM about internal network discovery activity. The only information you have is the IP address of the host performing the scan. Nothing else. The alert comes from your IDS logs.

What is the process of linking data from multiple sources to identify relationships between individual events called?

Correlation

What is the process of collecting and storing log data from multiple systems and sources into a single, unified location for easier analysis called?

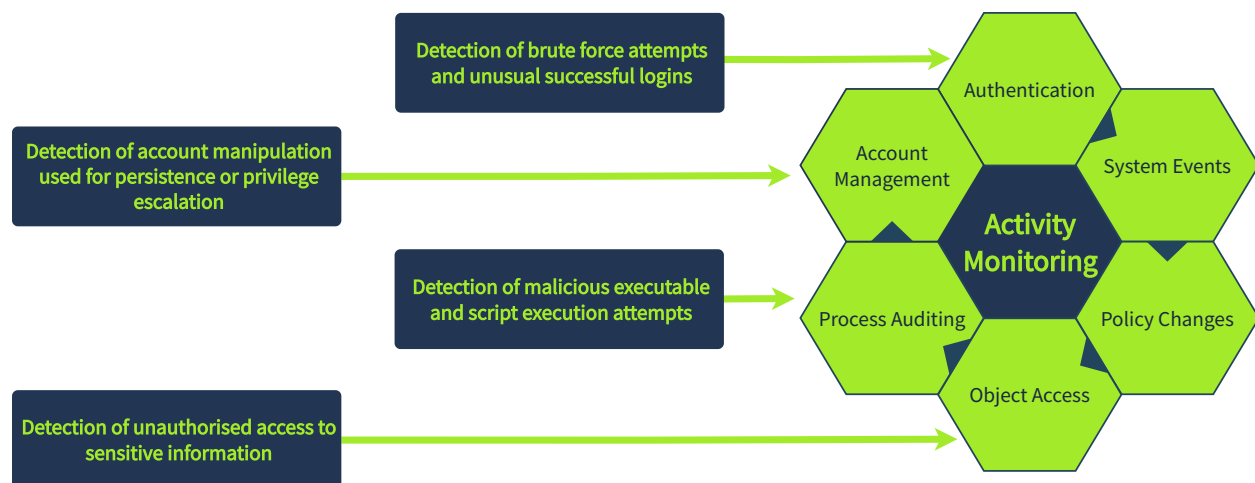
Centralisation

Task 3 - Log Sources Overview

Log Sources that an analyst can come across in SIEM and the value they can provide to the analyst. Every organisation has different types of resources from

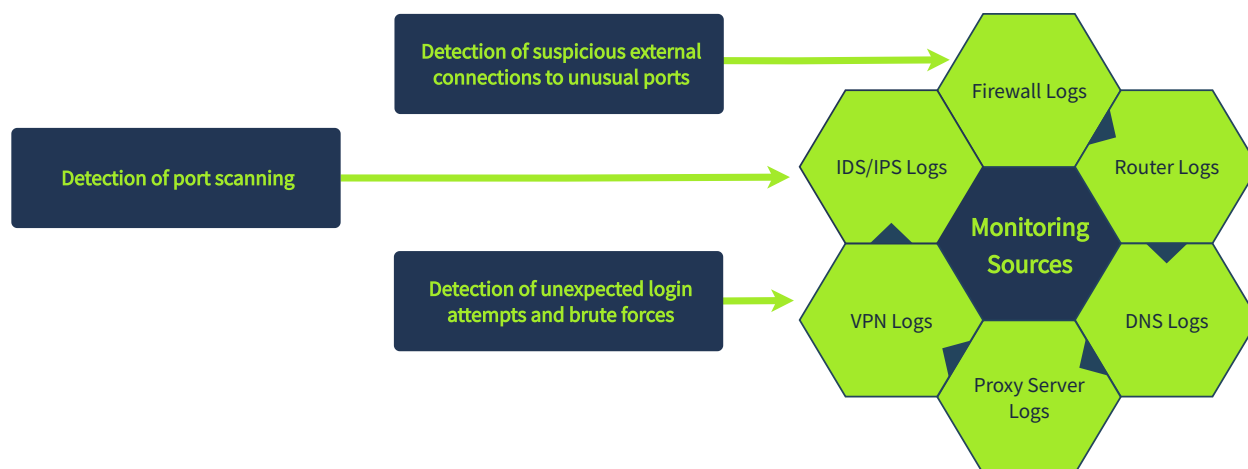
which logs can be collected into the SIEM, such as workstations, servers, network devices, identity providers, cloud services, various applications, and more. It can be helpful to think of this as a tree, where the organisation is the tree itself, and the roots represent the resources that make up this tree. While these components are often unique to each organisation, they all come together to form a logical structure. Logs from all or just a few of these resources are sent to the SIEM platform, where the analyst can use them for analysis and correlation events.

Host-Based Log Sources: Host-based log sources come from individual devices within the organisation, such as workstations and servers. Every organisation relies on workstations for employees to perform their duties and servers serving different purposes, such as web servers, SQL servers, DNS servers, and more. Typically, we monitor this type of behaviour from systems.



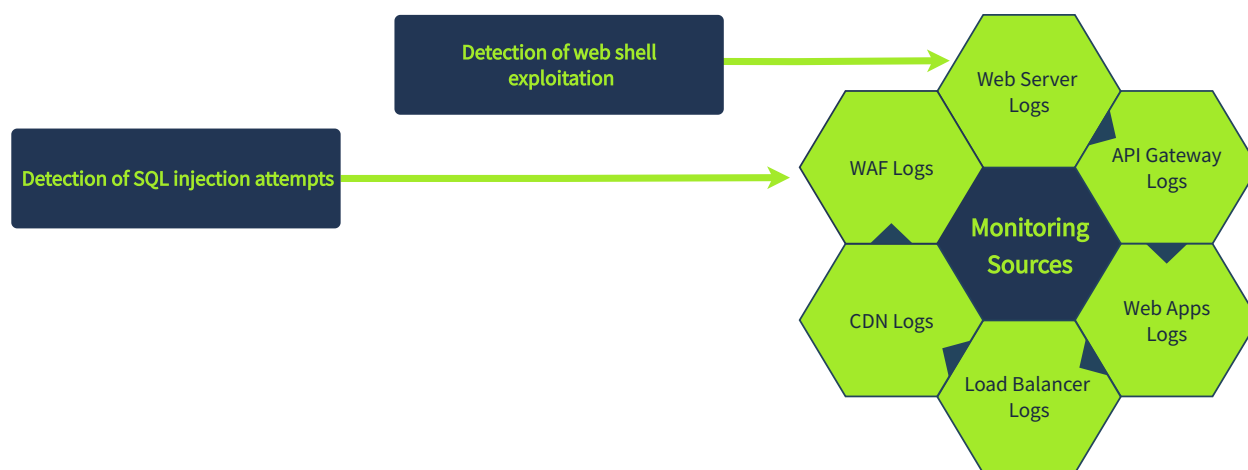
From all of these machines, a significant amount of data is sent to the SIEM for monitoring. This is not surprising, as nearly every attack will involve hosts in some way. Therefore, the data from these devices is invaluable during an investigation. We will dive deeper later into what can be discovered from this data.

Network-Based Log Sources: Network-based log sources collect data from network devices, such as firewalls, routers, IDS and IPS, and other systems. These devices play a critical role in monitoring traffic and connections across the network.



Just as the roots of a tree connect and support the branches, network logs give analysts visibility into how devices are communicating with each other. These logs can provide valuable insights when correlating data from different sources, making them a crucial part of a complete picture.

Web-Based Log Sources: And of course, almost every organisation has its own web application, and logs from it are also sent to the SIEM. It's through web vulnerabilities that attackers often gain access to an organisation's environment, so careful monitoring of web events is an essential part of every analyst's role in a SOC environment, day by day



Time Pitfalls: One common challenge analysts face when working in a SIEM is time, specifically how time is recorded across different log sources. Logs can come from systems in different time zones. Some can be in UTC, others in local

time, and some may not include a timezone at all. It's important to understand that your local time can be different from the time set in the SIEM.

Logs Normalisation: Different logs come in various formats. Some are in JSON, others in XML, or even plain text. Each system logs events in its own way, using different field names and structures. That's where normalisation comes in. The idea is to convert all these formats into a single, consistent structure, so that logs are easier to work with. Analysts can simply view everything in one standard way inside the SIEM. This makes searching, filtering, and linking events much easier. Without normalisation, every log source would feel like solving an entirely different puzzle.

What is the process of converting logs from different formats into a single format for easier analysis in a SIEM?

Normalisation

Which log source type can be used to detect the execution of a malicious script?

Host-Based

Task 4 - Windows Logs

When using a SIEM to analyse Windows logs, we usually talk about two main data sources: WinEventLogs and Sysmon. The second one must be installed and configured separately to start receiving logs. However, combining these two data sources actually provides clear visibility for analysing activity.

Sysmon: In some cases, even primary log source on Windows is Sysmon. It's an incredibly powerful place that logs a wide range of activity types, providing a high level of visibility during analysis. Using Sysmon logs, an analyst can observe a variety of activity types that provide deep visibility into system behaviour. It helps identify malicious process execution, network connections, possible process injection, registry changes, and file creation, among other activities.

Malicious Process Execution:

```
index=winenv EventCode=1 *powershell* AND *EncodedCommand*
```

```
| table _time ComputerName ParentUser ParentImage ParentCommandLine Image CommandLine
```

In this example, we detected that on host WINHOST05, a malicious `update_config.js` file was executed from the `C:\Users\Public` directory, which resulted in `cmd.exe` launching PowerShell with an encoded command.

New Search Close

```
1 index=winenv EventCode=1 *EncodedCommand* AND *powershell*
2 | table _time ComputerName User ParentImage ParentCommandLine Image CommandLine
```

✓ 1 event (before 8/13/25 7:44:28.000 AM) No Event Sampling

Events (1) Patterns Statistics (1) Visualization

100 Per Page Format Preview

_time	ComputerName	User	ParentImage	ParentCommandLine	Image	CommandLine
2025-08-11 10:58:27	WINHOST05	WINHOST05\ted-admin	C:\Program Files\nodejs\node.exe	"C:\Program Files\nodejs\node.exe" C:\Users\Public\update_config.js	C:\Windows\System32\cmd.exe	C:\Windows\System32\cmd.exe /d /s /c powershell.exe -NoP -W Hidden -EncodedCommand JABRAGUAcwB8ACAPQgACTAJAB1AG4QgABAEEXABQAEQXQBUEEAXABTAHRAKwB8AGUABQBLAGUAYQBwAHQAABVAHAZABHAHQZBYn

Annotations:

- 1. Victim Host
- 2. Compromised User Account
- 3. Launch Suspicious JS Script
- 4. Encoded PowerShell Execution

Suspicious Network Connection:

```
index=winenv EventCode=3 ComputerName=WINHOST05
| table _time ComputerName Image SourceIp SourcePort DestinationIp DestinationPort Protocol
```

As shown in the image below, a suspicious connection was initiated by the suspicious process `PPn423.exe` from the `Temp` folder, targeting the unusual port `9999` on IP address `83.222.191.2`. We also recommend checking this IP on TI platforms.

New Search Save As Create Table View Close

```
1 index=winenv EventCode=3 ComputerName=WINHOST05
2 | table _time ComputerName Image SourceIp SourcePort DestinationIp DestinationPort Protocol
```

✓ 1 event (before 8/13/25 9:00:50.000 AM) No Event Sampling

Events Patterns Statistics (1) Visualization

Show: 20 Per Page Format Preview: On

_time	ComputerName	Image	SourceIp	SourcePort	DestinationIp	DestinationPort	Protocol
2025-08-11 11:03:32	WINHOST05	C:\Windows\Temp\PPn423.exe	10.10.219.245	53530	83.222.191.2	9999	tcp

Annotations:

- 1. Victim Host
- 2. Malicious Process Establishing Network Connection
- 3. Victim IP Address
- 4. Destination Malicious IP, possible C2
- 5. Suspicious Destination Port
- 6. Network Communication Protocol

WinEventLogs: Windows Event Logs include a huge number of unique log files. While you're probably most familiar with the Security, System, and Application logs, in reality, the system contains over 200 different log channels beyond just those three.

A SOC analyst started a new shift and was handed information about suspicious activity on this host. Within just a couple of hours, another alert came in from the same machine, this time about a user account creation. This suggests the attacker is still present on the host. We need to review the Security logs to detect what's happening and share this information with our SOC L2 analyst.

The attacker likely decided to create a persistence mechanism in the form of a backup user account, which was created and enabled by ted-admin on the WINHOST05 host.



Windows System Logs: These logs record events generated by the operating system and its core services. They help detect various events related to services, system-level activity, and potential errors. These logs are an excellent place to look for potential persistence or privilege escalation attempts via services.

```
index=winenv EventCode=7045 OR EventCode=7036 ComputerName=WINHOST05
| table _time EventCode ComputerName Service_Name Service_Account Service_File_Name Mess
age
```

From the search results, we can see that on the host, a service named "User Updates" was created and started, which launches the malicious RNSfnsjdf.exe file from the Temp directory under the SYSTEM account. This is most likely a privilege escalation attempt, as we recall that the attacker previously only had access to the ted-admin account.

New Search

Save As

Create Table View

Close

1 index="winenv" EventCode=7045 OR EventCode=7036 ComputerName=WINHOST05

2 | table _time EventCode ComputerName Service_Name Service_Account Service_File_Name Message

All time

3 events (before 8/13/25 11:08:09.000 AM)

No Event Sampling

Job

Verbose Mode

Events (3) Patterns Statistics (3) Visualization

Show: 20 Per Page Format Preview: On

_time	EventCode	ComputerName	Service_Name	Service_Account	Service_File_Name	Message
2025-08-11 15:36:05	7045	WINHOST05	User Updates	LocalSystem	C:\Users\TOMBAR-1\AppData\Local\Temp\RNSfnsjdf.exe	A service was installed in the system.
2025-08-11 15:36:06	7036	WINHOST05	User Updates			Service Name: User Updates Service File Name: C:\Users\TOMBAR-1\AppData\Local\Temp\RNSfnsjdf.exe Service Type: user mode service Service Start Type: auto start Service Account: LocalSystem
2025-08-11 15:53:14	7036	WINHOST05	User Updates			The User Updates service entered the running state. The User Updates service entered the stopped state.

1. Service Created Event

2. Service Started/Stopped Events

3. Victim Host

4. Created Service Name

5. Service Executing User

6. Malicious Executable Triggered on Service Start

In that task, we examined a number of important log sources for an SOC L1 analyst and the value they can provide. Of course, there are many other logs that we will explore together in future rooms.

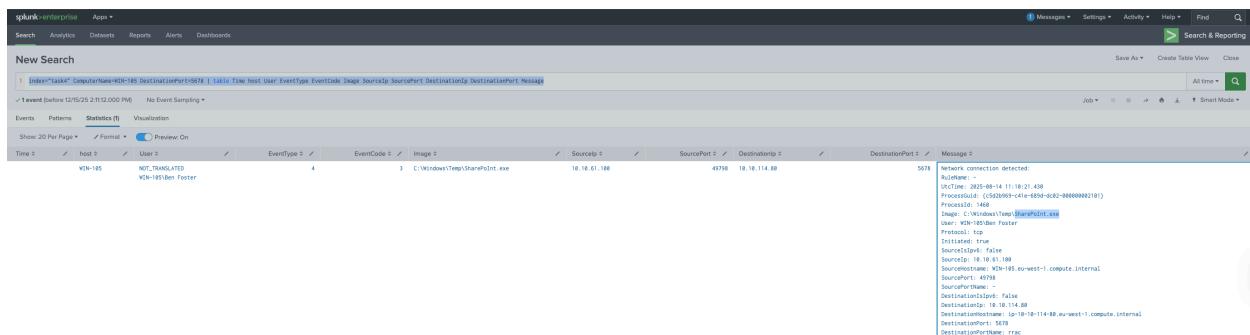
Practical Scenario: You are an SOC Level 1 Analyst on shift and have received an alert indicating a suspicious network connection using port 5678 on the WIN-105 host. Your task is to conduct an investigation and determine whether this activity is suspicious. The logs for this task are located in the Splunk index task4. Use the following query: `index=task4`

4.1: Which IP address was the connection established with?

Execute the query below to find the answer:

```
index="task4" ComputerName=WIN-105 DestinationPort=5678 | table Time host User EventType  
EventCode Image SourceIp SourcePort DestinationIp DestinationPort Message
```

Ans is: 10.10.114.80



The screenshot shows a Splunk search interface with the following query: `index="task4" ComputerName=WIN-105 DestinationPort=5678 | table Time host User EventType EventCode Image SourceIp SourcePort DestinationIp DestinationPort Message`. The results table displays a single event with the following details:

Time	host	User	EventType	EventCode	Image	SourceIp	SourcePort	DestinationIp	DestinationPort	Message
2025-08-14 11:10:22.691	WIN-105	WIN-105\Ben Foster	NET_TRANSLATED	4	C:\Windows\Temp\SharePoint.exe	10.10.81.100	49798	10.10.114.80	5678	Network connection detected RuleName: - InitTime: 2025-08-14 11:10:21.430 ProcessGuid: {c5d2b969-c4fe-689d-b0f6-000000000201} ProcessId: 1460 Image: C:\Windows\Temp\SharePoint.exe User: WIN-105\Ben Foster Protocol: tcp Initiated: true SourceCid: false SourceIp: 10.10.81.100 SourceProcessName: WIN-105-ku-west-1-compute.internal SourcePort: 49798 SourcePortName: - DestinationCid: false DestinationIp: 10.10.114.80 DestinationProcessName: ip-10-10-114-80-ku-west-1-compute.internal DestinationPort: 5678 DestinationPortName: rtmp

4.2: Which process initiated this suspicious connection?

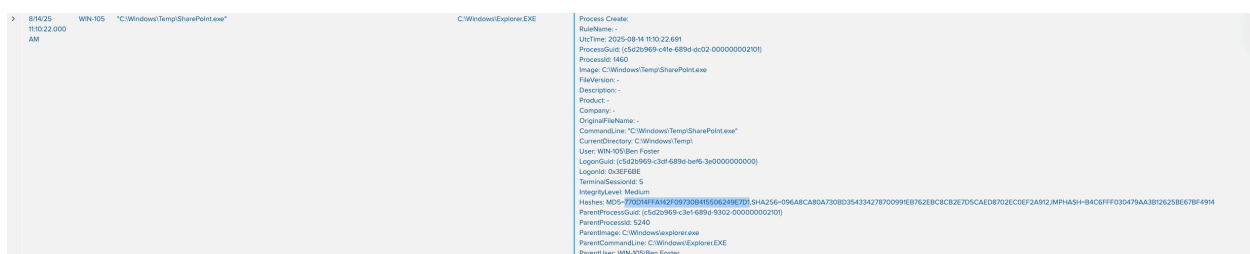
Ref to task 4.1 answer you can see `SharePoInt.exe` initiated the connection.

4.3: What is the MD5 hash of the malicious process from the previous question?

Well this one looked tricky but you can use the below query to find the MD5 hash of the process.

I started with this query:

```
index=task4 *SharePoInt*
```



The screenshot shows a Windows Event Viewer entry for a 'Process Create' event. The details pane displays the following information:

Property	Value
RuleName	-
InitTime	2025-08-14 11:10:22.691
ProcessGuid	{c5d2b969-c4fe-689d-b0f6-000000000201}
ProcessId	1460
Image	C:\Windows\Temp\SharePoint.exe
FileVersion	-
Description	-
Product	-
Company	-
OriginalFileName	-
CommandLine	"C:\Windows\Temp\SharePoint.exe"
CurrentDirectory	C:\Windows\Temp\
User	WIN-105\Ben Foster
LogonGuid	{c5d2b969-c4fe-689d-b0f6-000000000201}
LogonId	0x00000000
TerminalSessionId	5
IntegrityLevel	Medium
Hashes	MD5: 92098F8A25977984F5C624E7E55A4258-06A8CAB8A730BD35433427870099E762EBC8C3E705CAED702E0CF2AP2.RMP.HASH-B4C6FF930479AAB26258E67BF49H
ParentProcessId	{c5d2b969-c4fe-689d-b0f6-000000000201}
ParentProcessName	C:\Windows\explorer.exe
ParentCommandLine	C:\Windows\explorer.exe
ParentUser	WIN-105\Ben Foster

then I looked for process creation event and used the below query to get the MD5 hash.

```
index=task4 EventCode=1 Image="*SharePoInt.exe" | table _time ComputerName User EventType  
EventCode Image CommandLine Hashes
```

_time	ComputerName	User	EventType	EventCode	Image	CommandLine	Hashes
2025-08-14 11:18:22	WIN-105	WIN-105\Ben Foster	4	1	C:\Windows\Temp\SharePoInt.exe	C:\Windows\Temp\SharePoInt.exe	MD5=770D14FFA142F09730B415506249E7D1, SHA256=69646A8A73802543427878991037628C302E79542B76C0E724112, DPHASH=84C077F03A796A3B12628C57F4B14

Ans is: 770D14FFA142F09730B415506249E7D1

4.4: What is the name of the scheduled task that was created on the system?

I recall from task 01, there was sc query ran, so I searched for it using below query:

```
index=task4 *schtasks /create*
```

_time	host	CommandLine	ParentCommandLine	Message
2025-08-14 11:15:09.000 AM	WIN-105	schtasks /create /at once /at 15:30 /in "Office365 Install" /tr "C:\Windows\Temp\SharePoint.exe"	cmd.exe	Process Create: RunName: ... UseTime: 2025-08-14 11:15:09.849 ProcessId: 15428 Image: C:\Windows\System32\cmd.exe FileVersion: 10.0.17763.1601 (WinBuild.160108.0800) Description: Task Scheduler Configuration Tool Product: Microsoft Windows Operating System Company: Microsoft Corporation OriginalFilename: schtasks.exe CommandLine: schtasks /create /at once /at 15:30 /in "Office365 Install" /tr "C:\Windows\Temp\SharePoint.exe" CurrentDirectory: C:\Windows\Temp User: WIN-105\Ben Foster LogonId: 0x00000000-00000000-00000000-00000000 Upored: 0x00000000 TerminalSessionId: 5 IntegrityLevel: Medium Hashes: MD5=2F6C937AF205E4A9F643930CD49A734A256-48679CC4E0B4A8EDC243E2A4A8B3597A0D04A4E0E49D578C09F77C,MPHASH=8F0E8B9B94E30323D595A4C09 ParentProcessId: 5884 ParentImage: C:\Windows\System32\cmd.exe ParentCommandLine: cmd.exe ParentUser: WIN-105\Ben Foster

Ans is: Office365 Install

Task 5 - Linux Logs

When you're analysing Linux systems in a SIEM, you'll often start by looking at two key log sources. First, you'll see auth.log, which tracks authentication-related activity, such as user logins and sudo usage, among other things. These logs are crucial for spotting failed logins, unauthorised access attempts, or privilege escalation. Next, you'll come across syslog, which captures general system-level events. Here, you can monitor service restarts, cron jobs, and background processes - all helpful when building timelines or understanding how the system behaves over time. These tools significantly improve visibility.

Unusual login activities: As an SOC L1 analyst, you have received an alert about an unusual SSH login to the ubuntu username on the system. In this case, let's write a query to search for successful and failed login attempts to the system for the ubuntu user.

```
index=linux source="auth.log" *ubuntu* process=sshd
| search "Accepted password" OR "Failed password"
```

There were 97 events, and the last ones show successful login attempts to the ubuntu user. This can likely be classified as a successful brute-force attack - an activity that should be escalated to L2. From a learning perspective, let's take a look at what happened .

New Search Save As Create Table View Close

1 index=linux source="auth.log" *ubuntu* process=sshd
2 | search "Accepted password" OR "Failed password"

✓ 97 events (before 8/11/25 12:44:38.000 PM) No Event Sampling

Events (97) Patterns Statistics Visualization

Possible Brute-force attempt

Timeline format Zoom Out Zoom to Selection Deselect

Show Fields Format Show: 20 Per Page View: List

i	Time	Event
>	8/11/25 9:31:07.688 AM	2025-08-11T09:31:07.688251+00:00 deceptipot-demo sshd[1919]: Accepted password for ubuntu from 10.14.94.82 port 53600 ssh2 host = ce-splunk source = auth.log sourcetype = linux_secure
>	8/11/25 9:29:35.388 AM	2025-08-11T09:29:35.388871+00:00 deceptipot-demo sshd[1810]: Accepted password for ubuntu from 10.14.94.82 port 53567 ssh2 host = ce-splunk source = auth.log sourcetype = linux_secure
>	8/11/25 9:28:46.255 AM	2025-08-11T09:28:46.255540+00:00 deceptipot-demo sshd[1791]: Failed password for ubuntu from 10.14.94.82 port 53560 ssh2 host = ce-splunk source = auth.log sourcetype = linux_secure
>	8/11/25 9:28:46.255 AM	2025-08-11T09:28:46.255261+00:00 deceptipot-demo sshd[1792]: Failed password for ubuntu from 10.14.94.82 port 53561 ssh2 host = ce-splunk source = auth.log sourcetype = linux_secure
>	8/11/25 9:28:46.233 AM	2025-08-11T09:28:46.233027+00:00 deceptipot-demo sshd[1793]: Failed password for ubuntu from 10.14.94.82 port 53562 ssh2 host = ce-splunk source = auth.log sourcetype = linux_secure
>	8/11/25 9:28:46.185 AM	2025-08-11T09:28:46.185424+00:00 deceptipot-demo sshd[1794]: Failed password for ubuntu from 10.14.94.82 port 53563 ssh2 host = ce-splunk source = auth.log sourcetype = linux_secure
>	8/11/25 9:28:43.665 AM	2025-08-11T09:28:43.665931+00:00 deceptipot-demo sshd[1793]: Failed password for ubuntu from 10.14.94.82 port 53562 ssh2 host = ce-splunk source = auth.log sourcetype = linux_secure

Successful logins to ubuntu user from 10.14.94.82 IP

Failed logins to ubuntu user from 10.14.94.82 IP

Privilege Escalation behaviours Attackers often need root access on a system to gain access to certain files and more. Let's see if, in our case, there are any signs of such activities.

```
index=linux source="auth.log" *su*
| sort + _time
```

New Search Save As Create Table View Close

1 index=linux source="auth.log" *su*
2 | sort + _time All time Q

✓ 12 events (before 8/11/25 1:06:45.000 PM) No Event Sampling Job || ≡ ↶ ↷ ⌵ ⌴ Verbose Mode

Events (12) Patterns Statistics Visualization

✓ Timeline format > Show Fields ✓ Format Show: 20 Per Page View: List

i	Time	Event
>	11/8/25 13:43:08.608 PM	2025-08-11T13:43:08.608290+00:00 deceptipot-demo sudo: ubuntu : TTY=pts/0 ; PWD=/ ; USER=root ; COMMAND=/usr/bin/su host = ce-splunk source = auth.log sourcetype = linux_secure
>	11/8/25 13:43:08.612 PM	2025-08-11T13:43:08.612300+00:00 deceptipot-demo sudo: pam_unix(sudo:session): session opened for user root(uid=0) by ubuntu(uid=1000) host = ce-splunk source = auth.log sourcetype = linux_secure
>	11/08/25 13:43:08.618 PM	2025-08-11T13:43:08.618727+00:00 deceptipot-demo su[1460]: (to root) root on pts/1 host = ce-splunk source = auth.log sourcetype = linux_secure

← sudo su Command Execution

← Successful Root Session Initiation

← Confirmation of User Switch to Root

System Logs: System logs are another essential source of visibility. They capture events related to service activity, system restarts, and crons, all useful for identifying unusual system behaviour often linked to persistence or privilege escalation attempts.

Persistence Mechanisms: We hope you still remember our case in this task about the threat actor who successfully switched from ubuntu to root. In this scenario, syslog can also be useful, specifically for searching for activity related to persistence through cron jobs or services.

```
index=linux sourcetype=syslog ("CRON" OR "cron")
| search ("python" OR "perl" OR "ruby" OR ".sh" OR "bash" OR "nc")
```

We detected three interesting events. First, we can see that a suspicious pnr5433sw.sh file from the /tmp folder is being executed via cron every 5 minutes. Next, there are clear signs of a Perl reverse shell attempting to establish a connection to 10.10.101.12 IP on port 9999.

Ans is: 2025-08-12 09:52:57

5.2: Which user successfully escalated their privileges to root prior to the action from the first question?

Execute this query to find the answer:

```
index=task5  
| search sudo
```

Ans is: jack-brown

5.3: From which IP address did the user from the previous question successfully log in to the system?

Execute this query to find the answer:

```
index=task5  
| search jack-brown  
| search ssh
```

Ans is: 10.14.94.82

5.4: How many failed login attempts occurred prior to this successful login?

Execute this query to find the answer:

```
index=task5  
| search "Failed password for jack-brown"
```

ignore the message repeated 2 times lines

5.5: Which port is the persistence mechanism configured to connect to?

Execute this query to find the answer:

```
index=task5 *port*
```

The screenshot shows the Splunk Enterprise interface. At the top, there's a navigation bar with 'Search', 'Analytics', 'Dashboards', 'Reports', 'Alerts', and 'Dashboards'. Below this, a 'New Search' bar contains the query 'index=auth report'. The search results are displayed in a table format, showing various log entries. The table has columns for '_time', '_type', '_source', and '_raw'. The search results are filtered to show 18 events. The table shows log entries from 'deceptipat-demo' and 'deceptipat-demo' with various log levels and messages.

Ans is: 7654

Task 6 - Web Application Logs

Web Log Sources: Almost every organisation has its own website and therefore a web server running it. Logs from these web servers are collected from Apache, Nginx, and other resources. The most useful for analysis are the access logs, where you can detect requests to website resources. These logs often contain signs of malicious activity, such as scanning, DDoS attempts, various web-based attacks, and web shells. In addition, error logs can provide valuable insight, helping you understand possible failures or issues. Let's explore how different types of malicious activity can be detected using web access logs.

Brute Force Activity: Imagine you are a SOC L1 analyst on shift, and you've just received an alert about a possible brute-force attack targeting a WordPress login page. To investigate, we recommend starting by identifying the login page URL (e.g., /wp-login.php) and filtering for POST requests. Brute-force activity involves many repeated attempts, so set a threshold like count > 25 during 5 mins. Group the results by clientip to pinpoint the source of the activity. Other fields, such as user-agent, can also provide useful context. Below is the ready query.

```
index=* method=POST uri_path="/wp-login.php"
| bin_time span=5m
| stats values(referer_domain) as referer_domain values(status) as status values(useragent) as UserAgent values(uri_path) as uri_path count by clientip_time
```

```
| where count > 25
| table referer_domain clientip UserAgent uri_path count status
```

As a result, we can detect that in our example, the IP address 167.172.41.141 made as many as 160 requests to the wp-login.php page. Interestingly, the User-Agent string shows Hydra, a popular tool often used by attackers to perform brute-force attacks.



Possible Web Shell: Shortly after, you receive a second alert about possible web shell activity during your shift. To investigate potential web shell activity in Splunk, search for requests to script or executable file types such as .php, .asp, .jsp, or .exe, combined with POST and GET methods and a status=200 response. Web shells often generate a few suspicious requests in a short time frame, so set a threshold such as count > 2. Group the results by domain to identify patterns and review fields like clientip and user-agent for attacker fingerprints. Below is the ready query.

```
index=*
| search status=200 AND uri_path IN(*.php, *.phtm, *.asp, *.aspx, *.jsp, *.exe) AND (method=POST AND method=GET)
| stats values(status) as status values(useragent) as UserAgent values(method) as method values(uri) as uri values(clientip) as clientip count by referer_domain
| where count > 2
| table referer_domain count method status clientip UserAgent uri
```

As you can see in the results, we detect a variety of different URIs, but what caught our attention the most was 505.php, which could be a web shell. This means we need to take a closer look at it in more detail.

New Search Save As Create Table View Close

```

1 | index=*
2 | search status=200 AND uri_path IN(*.php, *.pht, *.asp, *.aspx, *.jsp, *.exe) AND (method=POST AND method=GET)
3 | stats values(status) as status values(useragent) as UserAgent values(method) as method values(uri) as uri values(clientip) as clientip count by referer_domain
4 | where count > 2
5 | table referer_domain count method status clientip UserAgent uri

```

Server error

✓ 336 events (before 8/11/25 7:50:52.000 AM) No Event Sampling

Events (336) Patterns Statistics (1) Visualization

Show: 20 Per

referer_domain	count	method	status	clientip	UserAgent	uri
http://demo-web.deceptitech.thm	14	GET POST	200	167.172.41.141	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/138.0.0.0 Safari/537.36	/wp-admin/admin-ajax.php /wp-admin/load-styles.php?c=0&dir=ltr&load%5Bchunk_0%5D=dashicons,admin-bar,code-editor,common,forms,admin-menu,dashboard,list-tables,edit,revisions,media,themes,about,nav-menus,wp-poi&load%5Bchunk_1%5D=nter,widgets,site-icon,110n,buttons,wp-auth-check,media-views&ver=6.8.1 /wp-admin/theme-editor.php?mode=edit

1. Target Domain

2. Upload and Access to Web Shell

3. Successful Exploitation

4. Attacker IP

5. Potential Web Shell

DDoS Activity: Your shift is almost over, but don't rush to close your laptop just yet, a new alert has just come in, likely the last one for today, about a possible DDoS attack. When checking for signs of a potential DDoS in Splunk, look for status code 503, which can mean the server is overloaded. Review for huge request counts in a short time, for example, more than 100 000 in 10 minutes filtered by IPs. Review the targeted domain, which user-agent was used, and the URI path to spot patterns or attacker fingerprints. Below is the ready query.

```

index=* status=503
| bin _time span=10m
| stats values(referer_domain) as referer_domain values(status) as status values(useragent) as UserAgent values(uri_path) as uri_path count by clientip _time
| where count > 100000
| table _time referer_domain clientip UserAgent uri_path count status

```

And from this example, we can see that the resource was unavailable for the last 10 minutes and received more than 1.5 million requests, which confirms a possible DDoS attack.

6.4: Which tool did the threat actor use?

Refer to user-agent string in the image.

wpscan

Task 7 - Conclusion

It provided valuable insights into how SIEM solutions can be used to detect and analyse malicious behaviour effectively. The hands-on experience with different log sources and scenarios enhanced my understanding of security operations and incident response.